Plan for a project for Robotics / camera agent interface where a virtual environment say a virtual park where a robot is walking and using imagejev it is deciding the direction.You can also suggest your opinion on this if you have any other good suggestion.

Yes—this is a strong project: build a **vision-language navigation (VLN) agent** in a simulated virtual park, where JEV-27B-VL receives the robot's camera view and chooses the next direction/action. JEV is designed for exactly this pattern: image + question + action options → calibrated probabilities, rather than free-form text.[1] [2]

Project goal

A robot walks through a virtual park toward a goal such as:
"Go to the fountain, avoid people, and stop at the bench."

At every step, the simulator renders a first-person camera image; JEV decides whether to move forward, turn left/right, stop, or avoid an obstacle. Vision-and-language navigation is a standard embodied-AI setup where an agent repeatedly chooses actions from visual observations and a language instruction.[3] [4]

Architecture

```
Virtual Park Simulator
        │
        ├── First-person RGB camera frame
        ├── Robot pose / battery / collision state
        └── Mission instruction
        │
        ▼
Agent Controller
        │
        ├── Builds action options
        ├── Calls JEV /v1/decide
        ├── Applies confidence threshold
        └── Sends action to simulator
        │
        ▼
Robot moves → new camera frame → repeat
```

JEV should be the **high-level decision layer**, while the simulator handles physics, collision detection, and low-level motion control. This separation—high-level decisions versus

continuous control—is also the standard approach for sim-to-real navigation systems. [4]

Tech stack

Layer	Suggested tools
Virtual environment	Unity + ML-Agents, Unreal Engine, Gazebo, or Webots
Robot simulation	ROS 2 / ROS bridge, or simulator-native physics
Camera	First-person RGB camera, optionally depth camera
Decision model	JEV-27B-VL served through vLLM's `/v1/decide` endpoint [1] [2]
Agent controller	Python FastAPI service or ROS node
UI	React/Next.js dashboard showing camera feed, map, chosen action, probabilities
Logging	SQLite/PostgreSQL + replayable episode logs

For a first version, I'd choose **Unity or Gazebo**: Unity is easier for a polished virtual park UI; Gazebo/ROS 2 is better if you may later transfer the system to a real robot.

Core decision loop

At each timestep:

1. Capture the robot's camera frame.
2. Define safe candidate actions: `forward`, `turn_left`, `turn_right`, `slow_down`, `stop`.
3. Send the image, mission, robot state, and action options to JEV.
4. Pick the action with the highest probability, subject to a confidence threshold.
5. Execute it in the simulator.
6. Log the decision, probabilities, and outcome for evaluation.

JEV's robotics example similarly makes small, repeated visual decisions—such as whether a target is left/right or above/below—then updates the robot's motion. [2]

Example decision request

```
import base64, requests

def image_data_url(path):
    with open(path, "rb") as f:
        return "" + base64.b64encode(f.read()).decode()

def decide(state, question, options):
    response = requests.post(
        "http://localhost:8000/v1/decide",
        json={
            "kind": "choice",
            "state": state,
            "question": question,
            "options": options,
```

```
        },
    )
    response.raise_for_status()
    data = response.json()
    return dict(zip(data["options"], data["probabilities"]))

state = [
    "Mission: Walk to the fountain without colliding with people.",
    "Robot state: battery 82%, no collision, speed normal.",
    "Camera view:",
    {"image": image_data_url("camera_frame.png")},
]

options = [
    "move_forward",
    "turn_left",
    "turn_right",
    "slow_down",
    "stop",
]

probs = decide(
    state,
    "Which action should the robot take next to reach the fountain safely?",
    options,
)

action = max(probs, key=probs.get)
print(action, probs)
```

JEV's `/v1/decide` supports choices among 2–256 options and returns a probability for each option in one forward pass.[2]

Safety and control rules

Do **not** let JEV directly command motor velocities. Instead:

- JEV selects a discrete high-level action.
- A local controller converts it into safe velocity commands.
- The simulator's collision sensor can veto unsafe actions.
- If JEV's top probability is below, say, 0.6, make the robot stop or ask for help.
- Keep an emergency-stop rule independent of the AI model.

This keeps the system robust when the model is uncertain or the camera view is ambiguous.

Useful missions

- **Waypoint navigation:** "Reach the playground entrance."
- **Obstacle avoidance:** "Avoid pedestrians and benches while moving forward."
- **Landmark search:** "Find the red mailbox and stop near it."
- **Patrol agent:** "Inspect all benches and report damaged ones."

- **Guided tour agent:** “Take visitors to the fountain, then the flower garden.”
- **Delivery robot:** “Carry a package from Gate A to the picnic area.”
- **Safety inspector:** “Identify litter, blocked paths, or unsafe playground equipment.”

My recommendation

Start with a **simple park grid world**, not a photorealistic 3D park. Use a small map, 4–6 action options, and missions such as “reach the fountain.” Once the loop works, add pedestrians, moving obstacles, multiple landmarks, and natural-language instructions.

I’d also add a **JEV-based evaluator**: after each episode, send the final camera frame, action history, and goal to JEV and ask whether the robot completed the mission. This gives you an automated success/failure judge for experiments. JEV has been used as an agent judge for completed versus failed trajectories.[5]

Success metrics

- Task success rate: percentage of episodes reaching the goal.
- Collision rate: collisions per 100 steps.
- Average steps to goal.
- Path efficiency: actual path length versus shortest path.
- Decision confidence and human-intervention rate.
- Generalization: performance on unseen park layouts and instructions.

Suggested milestones

1. **Week 1–2:** Build the virtual park, robot, camera, and manual keyboard control.
2. **Week 3–4:** Serve JEV locally and connect it to the simulator through `/v1/decide`.
3. **Week 5–6:** Implement the action loop, confidence threshold, and safety veto.
4. **Week 7–8:** Add obstacles, pedestrians, landmarks, and mission logging.
5. **Week 9–10:** Build the dashboard, run evaluations, and compare JEV decisions against a scripted baseline.

This design gives you a practical, safe, and demonstrable robotics/camera-agent project while keeping JEV in the role it is best suited for: fast visual action selection.[1] [2]

⁂

1. https://www.prnewswire.com/apac/news-releases/autotrust-ais-jev-27b-vl-tops-hugging-face-global-trending-list-as-open-decision-models-gain-momentum-302902296.html
2. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ
3. https://arxiv.org/html/2511.13524v2
4. https://www.mdpi.com/2076-3417/16/16/8050
5. https://github.com/DakePeng/Bonsai-JEV

6. https://huggingface.co/autotrust/JEV-27B-VL
7. https://huggingface.co/autotrust/JEV-27B-VL/blob/4000d2393be6718e604f8e7dca563a780ab78e78/README.md
8. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/blame/main/README.md
9. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/blob/44392aa19ff846f19a30b74137d501664862a3c5/README.md
10. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/blob/main/README.md
11. https://huggingface.co/blog/autotrust/autotrustjev-27b-vl-a-decision-model-that-learned
12. https://link.springer.com/article/10.1007/s10791-026-09977-z
13. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/commit/3396d0d7681bd2de48a2e129126365fd4ac64e91
14. https://huggingface.co/autotrust/JEV-27B-VL/blob/main/README.md
15. https://huggingface.co/autotrust/JEV-27B-VL/raw/main/README.md